

十大数据预处理方法总结

数据预处理在机器学习和数据科学中是非常非常重要的，因为它直接影响模型的性能和准确性。

- 提高数据质量:** 现实中的数据往往存在噪声、缺失值和异常值。如果不清洗和修正，这些问题会导致模型做出错误的预测或分析。因此，数据预处理通过清理、修复和标准化数据，确保输入的数据是干净、可靠的。
- 增强模型性能:** 不同的特征在数据集中可能有不同的量纲或尺度，这可能会使某些特征对模型的影响过大或过小。通过归一化、标准化和特征选择，可以让模型更均衡地利用所有特征，从而提高模型的性能。
- 防止过拟合:** 过多的特征或不相关的数据会增加模型的复杂性，导致过拟合。通过特征选择和降维，可以减少特征的数量，降低模型的复杂度，从而提高模型的泛化能力，防止过拟合。
- 处理不平衡数据:** 在实际应用中，数据集的类别分布可能严重不平衡，如果不加以处理，模型可能会偏向多数类，忽视少数类。通过过采样、降采样等方法可以平衡数据集，从而提高模型对少数类的识别能力。
- 提高模型的解释性和效率:** 简化数据结构和减少无用特征不仅可以提高模型的计算效率，还能使模型更容易解释。这样，决策者可以更好地理解模型的输出，并做出更明智的决策。

没有有效的数据预处理，模型可能难以捕捉到数据中的真实模式，最终影响预测结果的可靠性。

一、标准化 (Standardization)

1. 核心公式

标准化的目标是使数据具有零均值和单位方差。给定一个数据集，标准化的步骤如下：

计算均值:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

计算标准差:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

标准化公式:

$$z_i = \frac{x_i - \mu}{\sigma}$$

通过上述公式，我们将每个数据点转换为标准化值，使得所有数据点的均值为0，方差为1。

2. 优点

- 提高模型训练的稳定性。
- 使不同特征具有相同的尺度，适合于对距离敏感的算法。

3. 缺点

- 对于有明显离群值的数据，标准化可能会受到影响，导致结果不理想。

4. 适用场景

- 特别适用于要求数据分布近似正态分布的算法，如SVM、线性回归和KNN等。

5. 核心案例

```
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
california = fetch_california_housing()
data = pd.DataFrame(california.data, columns=california.feature_names)

# 原始数据统计信息
print("原始数据统计信息: \n", data.describe())

# 标准化
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# 将标准化后的数据转为DataFrame
data_scaled = pd.DataFrame(data_scaled, columns=california.feature_names)

# 标准化后数据统计信息
print("标准化后数据统计信息: \n", data_scaled.describe())

# 可视化前后数据分布
plt.figure(figsize=(16, 6))

# 标准化前的分布
plt.subplot(1, 2, 1)
sns.histplot(data['MedInc'], kde=True, color='blue')
plt.title('Distribution of Median Income before Standardization')

# 标准化后的分布
plt.subplot(1, 2, 2)
sns.histplot(data_scaled['MedInc'], kde=True, color='red')
plt.title('Distribution of Median Income after Standardization')

plt.show()
```

二、归一化 (Normalization)

1. 核心公式

归一化的目标是将数据缩放到特定的范围（通常是[0, 1]）。给定一个数据集，归一化的步骤如下：

找到最小值和最大值:

$$x_{\min} = \min(X)$$
$$x_{\max} = \max(X)$$

归一化公式:

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

通过该公式，所有数据点将被缩放到 [0, 1] 范围内。

2. 优点

- 使得数据在同一尺度上，有助于加快梯度下降的收敛速度。
- 适用于需要计算距离的算法，如KNN和神经网络。

3. 缺点

- 对于有离群值的数据，归一化后的数据分布可能会受到影响。

4. 适用场景

- 特别适用于距离度量敏感的算法，如KNN、神经网络。

5. 核心案例

我们将使用 Iris 数据集来展示归一化的效果，并生成相关的图表。

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
iris = load_iris()
data = pd.DataFrame(iris.data, columns=iris.feature_names)

# 原始数据统计信息
print("原始数据统计信息: \n", data.describe())

# 归一化
scaler = MinMaxScaler()
data_normalized = scaler.fit_transform(data)

# 将归一化后的数据转为DataFrame
```

```
data_normalized = pd.DataFrame(data_normalized, columns=iris.feature_names)

# 归一化数据统计信息
print("归一化后数据统计信息: \n", data_normalized.describe())

# 可视化前后数据分布
plt.figure(figsize=(16, 6))

# 归一化前的分布
plt.subplot(1, 2, 1)
sns.histplot(data['sepal length (cm)'], kde=True, color='green')
plt.title('Distribution of Sepal Length before Normalization')

# 归一化后的分布
plt.subplot(1, 2, 2)
sns.histplot(data_normalized['sepal length (cm)'], kde=True, color='orange')
plt.title('Distribution of Sepal Length after Normalization')

plt.show()
```

三、主成分分析 (PCA)

1. 核心公式

主成分分析的目标是将高维数据投影到低维空间，同时最大化投影后的方差。核心步骤包括：

1. **数据中心化:**

$$X_{\text{centered}} = X - \mu$$

其中 μ 是数据的均值向量。

2. **计算协方差矩阵:**

$$C = \frac{1}{n-1} X_{\text{centered}}^T X_{\text{centered}}$$

3. **特征分解:** 对协方差矩阵 C 进行特征分解，得到特征值和特征向量：

$$C \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

其中 λ_i 为特征值， $\mathbf{v}_i$ 为对应的特征向量。

4. **降维:** 选择前 k 个最大特征值对应的特征向量，形成投影矩阵 W_k ，然后将数据投影到低维空间：

$$Z = X_{\text{centered}} W_k$$

2. 优点

- 能有效降低数据的维度，减少计算复杂度。
- 可以去除冗余特征，提高模型的泛化能力。

3. 缺点

- PCA是线性方法，对于非线性数据效果不佳。
- 主成分的物理意义不明确，可能不利于解释。

4. 适用场景

- 特别适用于高维数据的降维，如图像处理、基因数据分析。

5. 核心案例

我们使用 `wine` 数据集来展示PCA的效果~

```
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
wine = load_wine()
data = pd.DataFrame(wine.data, columns=wine.feature_names)

# 原始数据统计信息
print("原始数据统计信息: \n", data.describe())

# PCA降维至2维
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data)

# 将PCA后的数据转为DataFrame
data_pca = pd.DataFrame(data_pca, columns=['PC1', 'PC2'])

# 可视化PCA结果
plt.figure(figsize=(10, 8))
sns.scatterplot(x='PC1', y='PC2', hue=wine.target, palette='bright',
data=data_pca)
plt.title('PCA of Wine Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```

四、标签编码 (Label Encoding)

1. 核心逻辑

标签编码的目标是将分类变量转换为整数值，使得模型可以处理非数值数据。

具体步骤如下：

类别映射：

假设有一个分类特征 $C = c_1, c_2, \dots, c_m$ ，标签编码将每个类别 c_i 映射为一个整数值 l_i 。

例如，'red' $\rightarrow$ 0, 'green' $\rightarrow$ 1, 'blue' $\rightarrow$ 2。

2. 优点

- 简单易用，适合有序类别数据。
- 对存储空间需求小，直接使用整数表示。

3. 缺点

- 对无序类别数据引入了虚假的顺序关系，可能影响模型性能。

4. 适用场景

- 适用于有序类别数据，如学历（高中、本科、硕士）。

5. 核心案例

我们使用模拟的学历数据来展示标签编码的效果，并生成相关的图表。

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns

# 创建示例数据
data = pd.DataFrame({
    'Education': ['High School', 'Bachelor', 'Master', 'PhD', 'Bachelor', 'High School', 'Master', 'PhD']
})

# 标签编码
label_encoder = LabelEncoder()
data['Education_encoded'] = label_encoder.fit_transform(data['Education'])

# 可视化标签编码结果
plt.figure(figsize=(8, 6))
sns.countplot(x='Education_encoded', data=data, palette='coolwarm')
plt.title('Label Encoded Education Levels')
plt.xlabel('Encoded Labels')
plt.ylabel('Count')
plt.show()

print("标签编码结果: \n", data)
```

五、独热编码（One-Hot Encoding）

1. 核心公式

独热编码的目标是将分类变量转换为二进制向量，使得每个类别被单独表示。假设有一个分类特征 $C = c_1, c_2, \dots, c_m$ ，独热编码的步骤如下：

类别向量化：

每个类别 c_i 被表示为一个长度为 m 的二进制向量，其中第 i 位为1，其余为0。

例如，'red' $\rightarrow [1, 0, 0]$ ，'green' $\rightarrow [0, 1, 0]$ ，'blue' $\rightarrow [0, 0, 1]$ 。

2. 优点

- 消除类别之间的顺序关系，避免模型引入虚假的顺序信息。
- 提高模型对类别变量的处理能力，适合无序类别。

3. 缺点

- 当类别较多时，会显著增加数据的维度，导致“维度灾难”。
- 对于高基数的分类特征，如城市、国家，编码后的稀疏矩阵会占用大量内存。

4. 适用场景

- 适用于无序类别数据，如性别、颜色等。

核心案例

我们使用 `Titanic` 数据集中的性别特征来展示独热编码的效果~

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
import matplotlib.pyplot as plt
import seaborn as sns

# 创建示例数据
data = pd.DataFrame({
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female', 'Male', 'Male',
              'Female']
})

# 独热编码
onehot_encoder = OneHotEncoder(sparse=False)
gender_encoded = onehot_encoder.fit_transform(data[['Gender']])

# 将独热编码后的数据转为DataFrame
gender_encoded_df = pd.DataFrame(gender_encoded,
                                  columns=onehot_encoder.categories_[0])

# 可视化独热编码结果
plt.figure(figsize=(8, 6))
sns.heatmap(gender_encoded_df, annot=True, cbar=False, cmap='coolwarm')
plt.title('One-Hot Encoded Gender')
plt.xlabel('Gender')
```

```
plt.ylabel('Sample Index')
plt.show()

print("独热编码结果: \n", gender_encoded_df)
```

六、过采样 (Over-sampling)

1. 核心公式

过采样的目标是通过复制少数类样本来平衡类别分布，主要方法包括简单复制和SMOTE（合成少数类过采样技术）。

SMOTE的核心步骤如下：

1. **选择样本**: 从少数类样本中随机选择一个样本 x 。
2. **找到k个最近邻样本**: 在少数类样本中找到k个最近邻样本 $x_{neighbor}$ 。
3. **生成新样本**: 在原样本 x 和一个最近邻样本 $x_{neighbor}$ 之间生成一个新样本 x_{new} ，其计算公式为：

$$x_{new} = x + \lambda \times (x_{neighbor} - x)$$

其中 λ 是一个0到1之间的随机数。

2. 优点

- 增加少数类样本数量，提高分类模型对少数类的识别能力。
- 使用SMOTE生成的新样本更具多样性，减轻了过拟合的风险。

3. 缺点

- 可能会引入噪声样本，导致模型过拟合。
- 简单复制方法可能会加剧少数类的过拟合问题。

4. 适用场景

- 特别适用于类别不平衡问题，如欺诈检测、医疗诊断等。

5. 核心案例

我们使用模拟的二分类不平衡数据集来展示过采样的效果~

```
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE
import matplotlib.pyplot as plt
import seaborn as sns

# 创建不平衡数据集
x, y = make_classification(n_samples=1000, n_features=2, n_informative=2,
                           n_redundant=0,
                           n_clusters_per_class=1, weights=[0.9, 0.1],
                           random_state=42)
```



```
# 使用SMOTE进行过采样
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# 可视化过采样前后的数据分布
plt.figure(figsize=(16, 6))

# 过采样前
plt.subplot(1, 2, 1)
sns.scatterplot(X[:, 0], X[:, 1], hue=y, palette='coolwarm')
plt.title('Before SMOTE')

# 过采样后
plt.subplot(1, 2, 2)
sns.scatterplot(X_resampled[:, 0], X_resampled[:, 1], hue=y_resampled,
palette='coolwarm')
plt.title('After SMOTE')

plt.show()
```

七、滑动窗口 (Sliding Window)

1. 核心公式

滑动窗口的目标是在时间序列数据上提取固定大小的子序列，用于捕捉局部时间特征。

核心步骤：

- 定义窗口大小:** 设定窗口大小 w 和步长 s 。
- 生成窗口:** 对时间序列数据 $X = x_1, x_2, \dots, x_n$ ，从第 t 个时间步开始，生成子序列 X_{window} ：
：

$$X_{\text{window}} = \{x_t, x_{t+1}, \dots, x_{t+w-1}\}$$

- 移动窗口:** 按照步长 s 移动窗口，生成下一个子序列，直到覆盖整个时间序列。

2. 优点

- 能有效捕捉时间序列中的局部模式或趋势。
- 适合短期预测和时间序列特征提取。

3. 缺点

- 选择合适的窗口大小较为困难。
- 对于长序列，滑动窗口生成的大量子序列可能会带来计算开销。

4. 适用场景

- 特别适用于时间序列预测，如股票价格预测、天气预报等。

5. 核心案例

我们使用模拟的股票价格数据来展示滑动窗口的效果~

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 创建模拟的股票价格数据
np.random.seed(42)
time = np.arange(100)
price = np.sin(time / 10) + np.random.normal(scale=0.1, size=len(time))

data = pd.DataFrame({'Time': time, 'Price': price})

# 定义滑动窗口函数
def sliding_window(data, window_size):
    return np.array([data[i:i + window_size] for i in range(len(data) -
window_size + 1)])

# 设置窗口大小
window_size = 10
windows = sliding_window(data['Price'], window_size)

# 可视化滑动窗口结果
plt.figure(figsize=(16, 6))
for i, window in enumerate(windows[:5]):
    plt.plot(range(i, i + window_size), window, marker='o')

plt.title('Sliding window on Simulated Stock Price')
plt.xlabel('Time')
plt.ylabel('Price')
plt.show()
```

八、插值 (Interpolation)

1. 核心公式

插值的目标是通过已知数据点估计未知数据点。常用方法包括线性插值、多项式插值和样条插值。

核心步骤:

1. **线性插值公式**: 对于两个已知数据点 (x_1, y_1) 和 (x_2, y_2) , 插值点 x 对应的 y 值为:

$$y = y_1 + \frac{(y_2 - y_1)}{(x_2 - x_1)} \times (x - x_1)$$

2. **多项式插值**: 利用拉格朗日插值公式或牛顿插值公式, 通过高阶多项式拟合已知数据点。
3. **样条插值**: 在每个子区间上使用低阶多项式插值, 同时保证多项式在区间连接处的光滑性。

2. 优点

- 能有效填补缺失数据, 提高数据集的完整性。
- 多种插值方法可用于不同数据特征的拟合。

3. 缺点

- 插值方法过于复杂时可能引入噪声或过拟合。
- 在外推时 (即在数据范围之外进行预测), 插值结果不可靠。

4. 适用场景

- 特别适用于有缺失值的数据, 如气象数据、传感器数据。

5. 核心案例

我们使用模拟的温度数据来展示插值的效果~

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

# 创建模拟的温度数据
np.random.seed(42)
time = np.arange(0, 100, 0.5)
temperature = np.sin(time) + np.random.normal(scale=0.1, size=len(time))

# 模拟缺失值
temperature[5:7] = np.nan

data = pd.DataFrame({'Time': time, 'Temperature': temperature})

# 线性插值
linear_interp = interp1d(data['Time'], data['Temperature'], kind='linear',
fill_value="extrapolate")
time_new = np.arange(0, 100, 0.1)
temperature_linear = linear_interp(time_new)

# 可视化插值结果
plt.figure(figsize=(12, 6))
plt.plot(data['Time'], data['Temperature'], 'o', label='Original Data')
```

```
plt.plot(time_new, temperature_linear, '-', label='Linear Interpolation')
plt.title('Linear Interpolation of Simulated Temperature Data')
plt.xlabel('Time')
plt.ylabel('Temperature')
plt.legend()
plt.show()
```

九、降采样 (Under-sampling)

1. 核心公式

降采样的目标是通过减少多数类样本来平衡类别分布。最简单的方法是随机采样，即从多数类中随机选择与少数类同等数量的样本。

计算多数类样本数: 假设多数类样本数为 N_{majority} ，少数类样本数为 N_{minority} 。

随机采样: 从多数类样本中随机选择 N_{minority} 个样本。

2. 优点

- 降低数据集的不平衡性，提高分类器对少数类的识别能力。
- 减少计算开销，适合大规模数据集。

3. 缺点

- 可能丢失多数类的关键信息，降低模型的整体性能。
- 只适用于多数类样本数远多于少数类样本数的情况。

4. 适用场景

- 特别适用于类别严重不平衡的数据，如垃圾邮件检测、不良信用记录分析等。

5. 核心案例

我们使用模拟的二分类不平衡数据集来展示降采样的效果~

```
from sklearn.datasets import make_classification
from imblearn.under_sampling import RandomUnderSampler
import matplotlib.pyplot as plt
import seaborn as sns

# 创建不平衡数据集
X, y = make_classification(n_samples=1000, n_features=2, n_informative=2,
                           n_redundant=0,
                           n_clusters_per_class=1, weights=[0.9, 0.1],
                           random_state=42)

# 使用随机降采样
undersampler = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = undersampler.fit_resample(X, y)

# 可视化降采样前后的数据分布
```

```
plt.figure(figsize=(16, 6))

# 降采样前
plt.subplot(1, 2, 1)
sns.scatterplot(X[:, 0], X[:, 1], hue=y, palette='coolwarm')
plt.title('Before Under-sampling')

# 降采样后
plt.subplot(1, 2, 2)
sns.scatterplot(X_resampled[:, 0], X_resampled[:, 1], hue=y_resampled,
palette='coolwarm')
plt.title('After Under-sampling')

plt.show()
```

十、特征选择 (Feature Selection)

1. 核心公式

特征选择的目标是从高维数据集中选出最具代表性的一部分特征，常用方法包括过滤法 (Filter)、包裹法 (Wrapper) 和嵌入法 (Embedded)。

以过滤法中的卡方检验为例，核心步骤：

1. **卡方检验公式**: 对于每个特征 X_i 与标签 y 之间，计算卡方统计量：

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

其中 O_i 是观察值， E_i 是期望值。

2. **特征选择**: 选择卡方统计量最大的前 k 个特征。

2. 优点

- 减少特征数量，降低模型复杂度，防止过拟合。
- 提高模型的泛化能力和解释性。

3. 缺点

- 可能丢失有用信息，影响模型性能。
- 需要精心选择特征选择方法，适应不同的模型和数据。

4. 适用场景

- 特别适用于高维数据集，如文本分类、基因表达数据分析。

5. 核心案例

我们使用 `Breast Cancer` 数据集来展示特征选择的效果~

```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, chi2
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
cancer = load_breast_cancer()
data = pd.DataFrame(cancer.data, columns=cancer.feature_names)

# 使用卡方检验选择前10个最重要的特征
selector = SelectKBest(chi2, k=10)
selector.fit(data, cancer.target)
data_selected = data.iloc[:, selector.get_support(indices=True)]

# 可视化特征选择结果
plt.figure(figsize=(12, 6))
sns.barplot(x=data.columns[selector.get_support()],
            y=selector.scores_[selector.get_support()])
plt.title('Top 10 Features Selected by Chi-Square Test')
plt.xlabel('Features')
plt.ylabel('Chi-Square Score')
plt.xticks(rotation=90)
plt.show()
```